



字符串

北京理工大学计算机学院
金旭亮

概述



在实际开发中，字符串可能是使用最广的一种数据类型了，因此，掌握字符串的使用方法，对于软件开发来说是很重要的。



在JDK中，字符串是String类型的实例，随着JDK版本的不断演进，String类型集成的功能越来越多，本讲就通过大量的实例，介绍String的基本使用技巧。



从面向对象的角度来说，字符串关联着相当丰富的面向对象技术与知识，从相对简单的“字符串”入手，对了解和掌握面向对象很有好处。

字符与字符串



字符：'Z'，'\n'等，按UNICODE进行编码，对应的数据类型为char，是Java 八种原始数据类型之一。

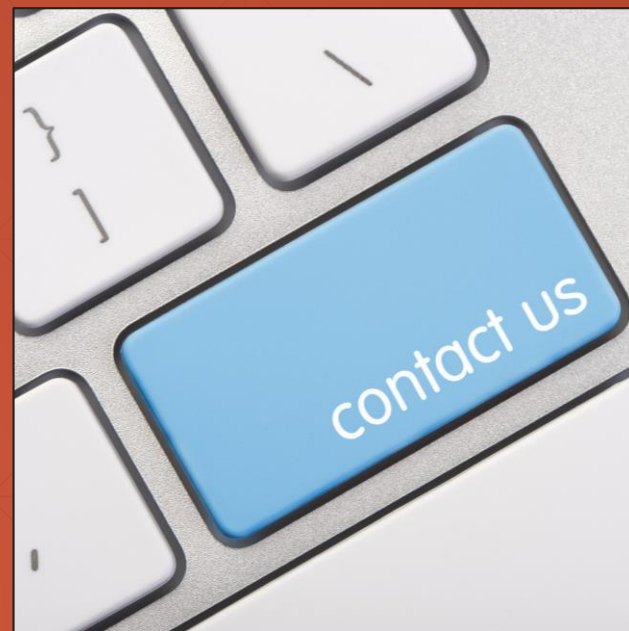


字符串："Hello"、"(201)13423422"等，是字符的集合，对应的数据类型为String



String：Java中它是一个**引用**类型。String类型的变量可以引用一个“字符串（String）”对象。

我们先来看看字符类型.....



热身：从字符类型开始

字符类型的基本用法

英文字符

```
static void charBasic() {  
    char c1 = 'A';  
    System.out.println(c1);  
    //char变量可以赋值给int变量  
    int charValue = c1;  
    //int变量中保存的实际上是Unicode码值  
    System.out.println(charValue);//65  
    //可以直接把一个整数传给char变量，这个整数被当成Unicode码值  
    char c2 = 65;  
    System.out.println(c2); //A  
    //可以直接使用==比较两个char变量的数值  
    System.out.println(c1 == c2);//true  
}
```

中文字符

```
static void chineseChar() {  
    char c1 = '中';  
    System.out.println(c1);  
    //char变量可以赋值给int变量  
    int charValue = c1;  
    //int变量中保存的实际上是汉字字符所对应的Unicode值  
    System.out.println(charValue);//20013  
    //可以直接把一个整数传给char变量，这个整数被当成Unicode码值  
    char c2 = (char)charValue;  
    System.out.println(c2); //中  
    //可以直接使用==比较两个char变量的数值  
    System.out.println(c1 == c2);//true  
}
```

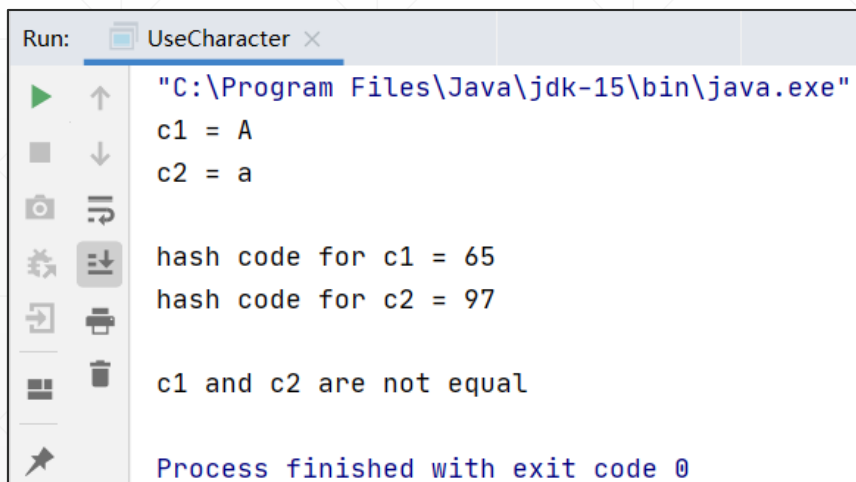
参看示例：UseChar.java



char 变量保存的数据占两个字节，被解释为特定字符的Unicode值，支持多国语言字符。

字符类型的包装类型-1

char对应着一个Character包装类型，封装了与字符操作相关的一些功能。



```
Run: UseCharacter x
"C:\Program Files\Java\jdk-15\bin\java.exe"
c1 = A
c2 = a

hash code for c1 = 65
hash code for c2 = 97

c1 and c2 are not equal

Process finished with exit code 0
```

```
static void characterBasic() {
    //两种创建Character对象的方法
    Character c1 = 'A';
    Character c2 = Character.valueOf( 'a' );

    //使用charValue方法将Character对象转换为char类型
    //使用toString()方法将Character对象转换为字符串
    String output =
        "c1 = " + c1.charValue() +
        "\nc2 = " + c2.toString() +
        "\n\nhash code for c1 = " + c1.hashCode() +
        "\n\nhash code for c2 = " + c2.hashCode();

    //比较两个Character对象的内容是否相等
    if ( c1.equals( c2 ) )
        output += "\n\nc1 and c2 are equal";
    else
        output += "\n\nc1 and c2 are not equal";

    System.out.println(output);
}
```

参看示例：UseCharacter.java

字符类型的包装类型-2

```
static String getCharInfo(char c) {  
    String info =  
        "是否在Unicode标准字符集中: " + Character.isDefined(c) +  
        "\n是否数字: " + Character.isDigit(c) +  
        "\n是否可作为Java标识符的第一个字符: " +  
        Character.isJavaIdentifierStart(c) +  
        "\n是否可作为Java标识符的其他字符: " +  
        Character.isJavaIdentifierPart(c) +  
        "\n是不是字母: " + Character.isLetter(c) +  
        "\n是字母或者数字: " +  
        Character.isLetterOrDigit(c) +  
        "\n是小写字母: " + Character.isLowerCase(c) +  
        "\n是大写字母: " + Character.isUpperCase(c) +  
        "\n转为大写: " + Character.toUpperCase(c) +  
        "\n转为小写: " + Character.toLowerCase(c);  
  
    return info;  
}
```



```
Run: UseCharacter x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
输入一个字符, 之后敲回车键: A  
是否在Unicode标准字符集中: true  
是否数字: false  
是否可作为Java标识符的第一个字符: true  
是否可作为Java标识符的其他字符: true  
是不是字母: true  
是字母或者数字: true  
是小写字母: false  
是大写字母: true  
转为大写: A  
转为小写: a  
  
Process finished with exit code 0
```

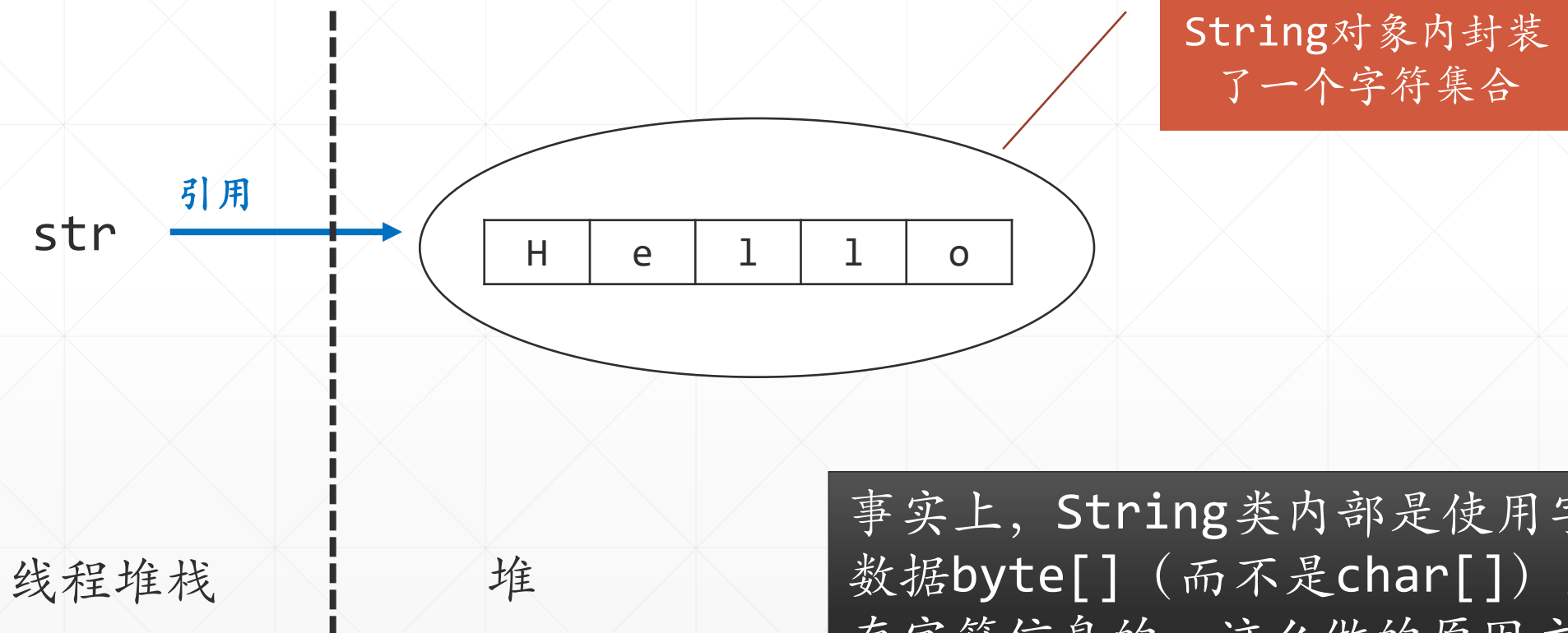
参看示例: UseCharacter.java



正式进入字符串的世界

String内存模型

```
String str = "Hello" ;
```



事实上，String类内部是使用字节数据`byte[]`（而不是`char[]`）来保存字符信息的，这么做的原因主要是为了尽可能少地占用内存。

String的构造方法

```
// 字符串对象的多种构造方法
s1 = new String();    // 缺省构造方法, 可被简化为直接赋值空串
s2 = new String(s);   // 拷贝构造方法
s3 = new String(charArray); // 以字符数组初始化字符串
// 从字符数组第6个元素起取3个字符
s4 = new String(charArray, offset: 6, count: 3);
s5 = new String(byteArray); // 以字节数组初始化字符串
// 从字节数组第4个元素起取4个字节
s6 = new String(byteArray, offset: 4, length: 4);
// 从StringBuffer对象中提取字符数据创建字符串 (注意, 是复制)
s7 = new String(buffer);
// 使用valueOf()系列方法, 可以很方便地将各种类型的数据直接转换为字符串
// 将字符数组转换为字符串
s8 = String.valueOf(charArray);
// 将double数值转换为字符串
s9 = String.valueOf(100.0d);
```

构造方法在创建对象时自动执行。

同一个类中可以有多多个构造方法, 它们构成方法重载关系。

String类就是一个很典型的实例。

参看示例: UseString.java

字符串转换为字符数组

//从字符串中提取字符数组

```
static void stringAndArray(){  
    String str="你好，中国！";  
    //将整个字符串中的所有字符都提取出来  
    char[] charArray = str.toCharArray();  
    //输出字符数组的内容  
    printStringArray(charArray);  
    char[] chars=new char[2];  
    //从字符串中提取字符，构建字符数组  
    //四个参数的含义  
    //1.被拷贝字符在字符串中的起始位置  
    //2.被拷贝的最后一个字符在字符串中的下标再加1  
    //3.目标字符数组  
    //4.拷贝的字符放在字符数组中的起始下标  
    str.getChars( srcBegin: 3, srcEnd: 5, chars, dstBegin: 0 );  
    printStringArray(chars);  
}
```

//打印字符数组的内容

```
static void printStringArray(char[] chars){  
    String output="";  
    for ( int i = 0; i < chars.length;i++ )  
        output += "["+i+"] "+ chars[ i ] +"\n";  
    System.out.println(output);  
}
```

运行结果

[0] 你
[1] 好
[2] ,
[3] 中
[4] 国
[5] !

[0] 中
[1] 国

字符串的拼接

```
//重复拼接子串，生成新字符串
static void stringRepeat() {
    String a5 = "a".repeat(5);
    //生成: aaaaa
    System.out.println(a5);
}
```

```
//字符串的拼接
static void stringConcat() {
    String s1 = "Happy ", s2 = "Birthday";
    //使用concat方法将两个字符串连接为一个新的字符串
    System.out.println(s1.concat(s2)); //Happy Birthday
    //也可以使用"+"达到相同的目的
    System.out.println(s1 + s2); //Happy Birthday
    //"+"运算符可以将字符串与任意一个对象相拼接
    System.out.println("99+1="+(99+1)); //99+1=100
    //当前时间: 2020-10-01
    System.out.println("当前时间: "+ LocalDate.now());
}
```



拼接之后，得到的是一个新的字符串，并不是修改原有的字符串实现的。

填充字符串

使用String.format方法，可以定义一个字符串模板，在运行时，将特定变量的值“填入”到模板中，动态地构建出一个字符串。

```
static void stringFill() {  
  
    var item = "Shirt";  
    var size = "M";  
    var price = 14.99;  
    var color = "Red";  
  
    var template = "Clothing item: %s, size %s, color %s, $%.2f";  
    var itemString :String = String.format(template,  
        item, size, color, price);  
    //Clothing item: Shirt, size M, color Red, $14.99  
    System.out.println(itemString);  
}
```



String.format()方法的第一个参数是字符串模板，其中%打头的是可以被“填入”值的“空”，%s表示这里需要一个字符串，%.2f表示这里需要一个保留两位小数的float数值。

移除空白字符

```
//移除首尾的空白字符 (JDK11)
static void stringStrip() {
    String example = "  前后都有空白字符的字符串  ";
    System.out.println("|" + example.strip() + "|");
    System.out.println("|" + example.stripLeading() + "|");
    System.out.println("|" + example.stripTrailing() + "|");
}
```



|前后都有空白字符的字符串|
|前后都有空白字符的字符串 |
| 前后都有空白字符的字符串|

在实际开发中，遇到需要用户从键盘输入的场景时，建议调用`strip()`方法移除字符前后的空白字符。

段落缩进

```
//JDK 12: 构建多行缩进的字符串
static void stringIndent() {
    String star = "*";
    for (int i = 0; i < 5; i++) {
        System.out.print(star.indent(i));
    }
    for (int i = 4; i >= 0; i--) {
        System.out.print(star.indent(i));
    }
}
```



```
*
 *
  *
   *
    *
   *
  *
 *
*
```



使用JDK 12新增的indent方法，可以很方便地格式化字符串，经常用于输出程序代码、XML或HTML文档之类需要分级缩进的内容。

多行文本块—TextBlock

```
//使用连续三个双引号，定义一个多行文本块
String poem = """
        题临安邸
        (宋) 林升
        山外青山楼外楼，西湖歌舞几时休。
        暖风熏得游人醉，直把杭州作汴州。
        """;

//输出整首诗歌
System.out.println(poem.stripIndent());
```

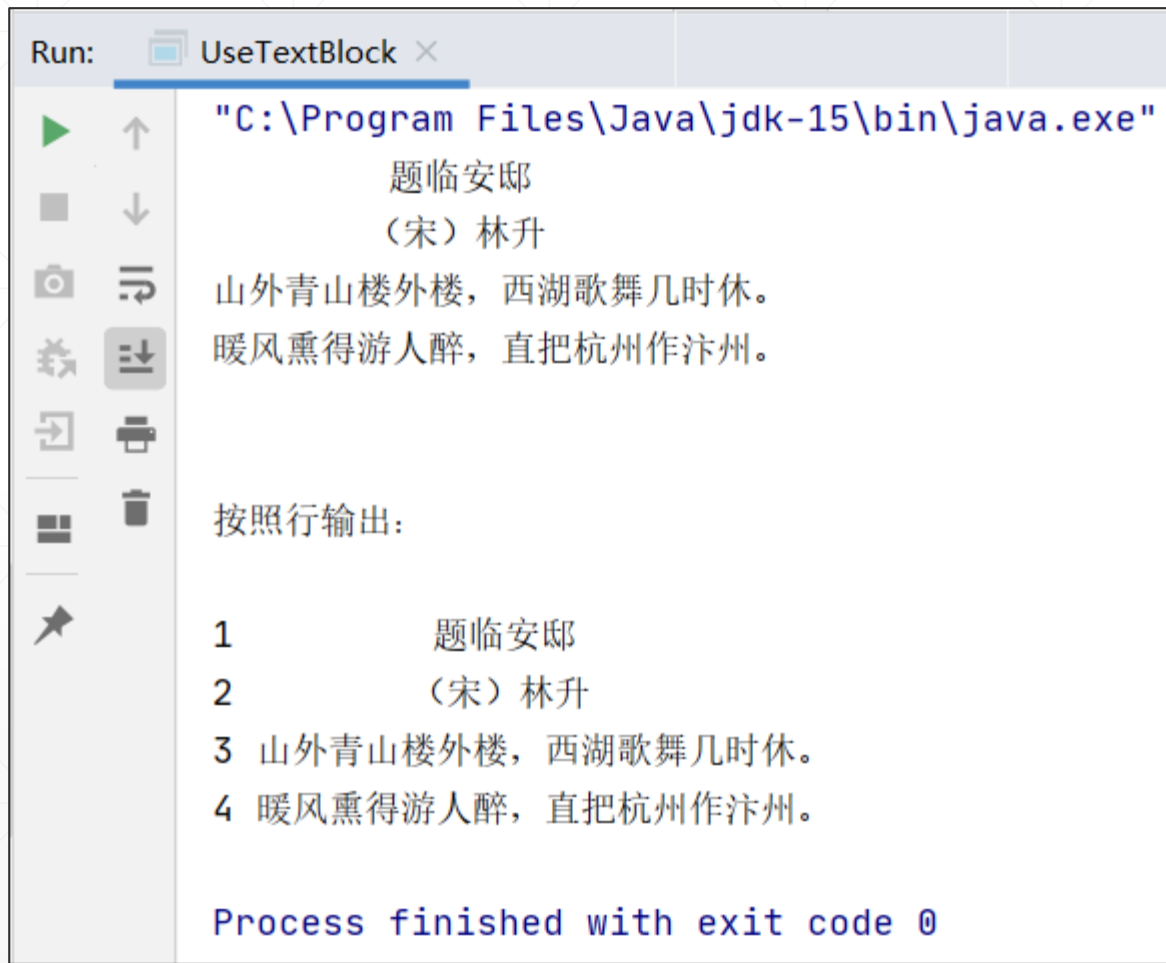
```
System.out.println("\n按照行输出: \n");
//JDK11为String类添加了一个lines()方法，
//此方法返回一个Stream<String>对象，
//可以把它看成是一个行的集合，
//这里用到了Lambda表达式将其按行输出
//Lambda表达式中的用于生成行号的变量i
//是类中定义的一个静态变量
poem.lines().forEach(line -> {
    i++;
    System.out.println(i + " " + line);
});
```

参看示例：UseTextBlock.java



多行文本块特别适合于输出大段文字。

多行文本块示例的输出结果



Run: UseTextBlock x

"C:\Program Files\Java\jdk-15\bin\java.exe"

题临安邸
(宋) 林升

山外青山楼外楼，西湖歌舞几时休。
暖风熏得游人醉，直把杭州作汴州。

按照行输出：

```
1      题临安邸
2      (宋) 林升
3 山外青山楼外楼，西湖歌舞几时休。
4 暖风熏得游人醉，直把杭州作汴州。
```

Process finished with exit code 0

注意：多行文本块是JDK 15才成为正式特性的

Project language level:

This language level is default for all project modules.

A module specific language level can be configured for each of the modules as required.

SDK default (15 - Text blocks)

11 - Local variable syntax for lambda parameters

12 - No new language features

13 - No new language features

14 - Switch expressions

14 (Preview) - Records, patterns, text blocks

15 - Text blocks

15 (Preview) - Sealed types, records, patterns, local enums and interfaces

X - Experimental features

多行文本块也支持动态填充

```
private static void fillTextBlock(String strValue, int intValue) {  
    String textBlock = ""  
        |           | 从外部传入的字符串参数值: %s,  
        |           | int参数值: %d  
        |           | "";  
    textBlock = textBlock.formatted(strValue, intValue);  
    System.out.println(textBlock);  
}
```

调用:

```
fillTextBlock( strValue: "Hello", intValue: 1000);
```

从外部传入的字符串参数值: Hello,
int参数值: 1000

“空串”的判断方式

```
static void stringIsBlankOrEmpty() {  
    var str = ""; //空串, 其长度为0  
    //isBlank: JDK11引入, 用于检测字符串是否仅由  
    // white space(即空白字符)构成  
    System.out.println(str.isBlank()); //true  
    //isEmpty(), JDK 6引入, 仅用于检测空串  
    System.out.println(str.isEmpty()); //true  
    str = "   "; //全部由空格组成的字符串  
    System.out.println(str.isBlank()); //true  
    System.out.println(str.isEmpty()); //false  
    //Tab, 也算作空白字符  
    str = "    ";  
    System.out.println(str.isBlank()); //true  
    System.out.println(str.isEmpty()); //false  
    str = "\n"; //用于换行的格式控制字符, 也算是空白符  
    System.out.println(str.isBlank()); //true  
    System.out.println(str.isEmpty()); //false  
}
```

注意区分“空串”和仅由
“空白符”构成的字符串。

动手动脑



请运行以下示例代码，查看其输出结果。如何解释这样的输出结果？从中你能总结出什么？

```
static void stringPool() {  
    String s0 = "Hello";  
    String s1 = "Hello";  
    String s2 = "He" + "llo";  
    System.out.println(s0 == s1); // true  
    System.out.println(s0 == s2); // true  
  
    String s3 = new String( original: "Hello");  
    String s4 = new String( original: "Hello");  
    System.out.println(s3 == s4); // false  
}
```

示例分析参考结论



在Java中，内容相同的字符串常量（“Hello”）只保存一份以节约内存，所以s0,s1,s2实际上引用的是同一个对象。



编译器在编译s2一句时，会去掉“+”号，直接把两个字符串连接起来得一个字符串（“Hello”）。这种优化工作由Java编译器自动完成。



当直接使用new关键字创建字符串对象时，虽然值一致（都是“Hello”），但仍然是两个独立的对象。

再试.....



```
static void stringAssign() {  
    String s1 = "a";  
    String s2 = s1;  
    System.out.println(s1 == s2); //true  
    s1 += "b";  
    System.out.println(s1 == s2); //false  
    System.out.println(s1 == "ab"); //false  
    System.out.println(s1.equals("ab")); //true  
}
```

为什么会有上述的输出结果？从中你又能总结出什么？

示例分析参考结论



“==”运算符用于比较两个字符串变量是否引用同一个字符串对象。



示例中用s1给字符串变量s2赋值意味着：两个变量(s1,s2)现在引用同一个字符串对象“a”，所以，`s1 == s2` 返回 `true`。



String对象的内容是只读的，示例中使用“+=”修改s1变量的值，实际上是得到了一个新的字符串对象，其内容为“ab”，而s2引用的还是原来的“a”，所以，这时，`s1 == s2` 返回 `false`。



另外，代码中的“ab”字符串是一个常量，它所代表的字符串对象与使用“+=”生成的并由s1所引用的“ab”对象无关，是两个独立的对象。



`String.equals()`方法可以比较两个字符串的内容（区分大小写），类似地，有一个`equalsIgnoreCase()`方法，比较时不理睬大小写。

字符串的hashCode

```
static void stringHashCode() {  
    String s1 = "hello",  
           s2 = "Hello",  
           s3 = new String(s2);  
    //hello的hashCode=99162322  
    System.out.println(s1 + "的hashCode=" + s1.hashCode());  
  
    //s2与s3引用不同的字符串对象  
    System.out.println(s2 == s3); //false  
    //s2与s3所引用字符串对象的值相等  
    System.out.println(s2.equals(s3)); //true  
  
    //Hello的hashCode=69609650  
    System.out.println(s2 + "的hashCode=" + s2.hashCode());  
    //Hello的hashCode=69609650  
    System.out.println(s3 + "的hashCode=" + s3.hashCode());  
}
```

只要两对象的“内容”相等，其hashCode值也应该相等

动手动脑



请课后阅读 `String.equals()` 方法的源代码，搞清楚JDK中是怎样比较两个字符串的“内容”的。

请课后阅读 `String.hashCode()` 方法的源代码，搞清楚JDK中是怎样依据字符串的“内容”生成hashCode的。

字符串比较-1

```
var s1 = new String( original: "hello");  
var s2 = new String( original: "good bye");  
var s3 = new String( original: "Happy Birthday");  
var s4 = new String( original: "happy birthday");
```

//使用字典法进行比较, 返回0表两字符串相等, 小于返回负值, 大于返回正值

```
var output =
```

```
    "\ns1.compareTo( s2 ) is " + s1.compareTo(s2) +  
        "\ns2.compareTo( s1 ) is " + s2.compareTo(s1) +  
        "\ns1.compareTo( s1 ) is " + s1.compareTo(s1) +  
        "\ns3.compareTo( s4 ) is " + s3.compareTo(s4) +  
        "\ns4.compareTo( s3 ) is " + s4.compareTo(s3) +  
        "\n\n";
```

```
System.out.println(output);
```



```
s1.compareTo( s2 ) is 1  
s2.compareTo( s1 ) is -1  
s1.compareTo( s1 ) is 0  
s3.compareTo( s4 ) is -32  
s4.compareTo( s3 ) is 32
```

compareTo: 使用字典法进行比较, 返回0表两字符串相等, 小于返回负值, 大于返回正值。

字符串的比较

regionMatches: 比较两字符串中的某一部分是否相等。

```
//比较两字符串中的某一部分是否相等
//regionMatches方法参数的含义如下:
//1.调用这个方法的字符串中的起始下标
//2.要比较的字符串
//3.要比较的字符串的起始下标
//4.要比较的字符串比较的字符长度
if (s3.regionMatches( toffset: 0, s4, ooffset: 0, len: 5))
    System.out.println("s3 和 s4 的前5个字符匹配\n");
else
    System.out.println("s3 和 s4 的前5个字符不匹配");
```

→ s3 和 s4 的前5个字符不匹配

字符串查找-1

在字符串查找字符或子串，调用 `indexOf` 和 `lastIndexOf` 方法即可。

```
static void stringIndex() {  
    String letters = "abcdefghijklmabcdefghijklm";  
    System.out.println(letters.indexOf('c')); //2  
    //第2个参数是开始查找的起始下标  
    System.out.println(letters.indexOf(ch: 'c', fromIndex: 1)); //2  
    System.out.println(letters.indexOf('$')); // -1  
    System.out.println(letters.lastIndexOf(ch: 'c')); //15  
    System.out.println(letters.lastIndexOf(ch: 'm', fromIndex: 5)); // -1  
}
```

字符串查找-2

查询字符串是否以某字符串开头和结尾：startsWith 和 endsWith方法。

```
static void stringStartEnd() {  
    String[] strings = {"started", "starting",  
                        "ended", "ending"};  
    for (String string : strings) {  
        if (string.startsWith("st"))  
            System.out.println(string + "以st开头");  
    }  
    for (String string : strings) {  
        if (string.endsWith("ed"))  
            System.out.println(string + "以ed结尾");  
    }  
}
```



```
Run: UseString x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
started以st开头  
starting以st开头  
started以ed结尾  
ended以ed结尾  
  
Process finished with exit code 0
```

提取子串

提取子串，调用subString方法：

```
static void stringSlice() {  
    String letters = "abcdefghijklmabcdefghijklm";  
    //输出: hijklm  
    System.out.println(letters.substring(20));  
    //输出: bcdef  
    //(起始下标,要拷贝子串的最后下标加1但不包括该下标)  
    System.out.println(letters.substring(1, 6) );  
}
```



那些可以修改的字符串

可以修改的字符串



前面我们已经知道，String对象创建后，它的内容是不可改的，如果确实需要一个可以修改内容的字符串，可以使用StringBuffer。



StringBuffer对象其内容可以修改，其占用的空间能自动增长



可以调用StringBuffer.append()方法不断地向StringBuffer对象“追加”新的内容，追加完毕之后，再调用它的toString()方法得到最终的字符串。在整个字符串构建过程中，都是在同一块内存区域中完成的。

StringBuffer对象的创建

```
static void stringBufferConstructors() {  
    //常用的三种StringBuffer构造方法  
    var buf1 = new StringBuffer();  
    var buf2 = new StringBuffer( capacity: 10);  
    var buf3 = new StringBuffer("hello");  
  
    //使用toString()方法将字符串缓冲区中的字符提取出来  
    System.out.println(buf1.toString());  
    System.out.println(buf2.toString());  
    System.out.println(buf3.toString());  
}
```

向StringBuffer中追加数据

```
static void stringBufferAppend() {  
    var buf = new StringBuffer();  
    //append()方法有多种重载形式, 可接收多种类型的数据  
    buf.append("hello\n");  
    char[] charArray = {'a', 'b', 'c', 'd', 'e', 'f'};  
    buf.append(charArray);  
    buf.append("\n");  
    buf.append(charArray, offset: 0, len: 3);  
    //append()方法支持级联调用  
    buf.append("\n").append(true).append("\n")  
        .append('Z').append("\n").append(100);  
    //输出最终结果  
    System.out.println(buf.toString());  
}
```



```
hello  
abcdef  
abc  
true  
Z  
100
```

支持“级联调用”的奥秘

StringBuffer.append() 方法源码，揭示了append() 方法支持级联调用的秘密.....

```
@Override
@HotSpotIntrinsicCandidate
public synchronized StringBuffer append(String str) {
    toStringCache = null;
    super.append(str);
    return this;
}
```

操作缓冲区中的字符

```
static void stringBufferChars() {  
    var buf = new StringBuffer("hello there");  
    System.out.println("\nCharacter at 0: " + buf.charAt(0));  
    buf.setCharAt(index: 0, ch: 'H');  
    buf.setCharAt(index: 6, ch: 'T');  
    System.out.println(buf.toString());  
    //提取字符串缓冲区中的字符到字符数组中  
    var charArray = new char[buf.length()];  
    //(起始下标,长度,字符数组名,起始下标)  
    buf.getChars(srcBegin: 0, buf.length(), charArray, dstBegin: 0);  
    for (int i = 0; i < charArray.length; ++i)  
        System.out.print(charArray[i] + ",");  
    System.out.println();  
    //reverse: 倒转字符串  
    buf.reverse();  
    System.out.println(buf.toString());  
}
```



```
Character at 0: h  
Hello There  
H,e,l,l,o, ,T,h,e,r,e,  
erehT olleH
```

StringBuffer的容量与长度

```
var buf = new StringBuffer("Hello,");  
  
//length = 6  
System.out.println("length = " + buf.length());  
//capacity = 22  
System.out.println("capacity = " + buf.capacity());  
//追加一个新的字符串  
buf.append("how are you?");  
//输出: Hello,how are you?  
System.out.println(buf.toString());  
//length = 18  
System.out.println("length = " + buf.length());  
//capacity = 22  
System.out.println("capacity = " + buf.capacity());
```

```
//ensureCapacity:保证StringBuffer的最小容量  
buf.ensureCapacity(minimumCapacity: 75);  
//New capacity = 75  
System.out.println("New capacity = " + buf.capacity());  
  
//setLength:增大或减小StringBuffer的长度  
buf.setLength(10);  
System.out.println("New length = " + buf.length());  
//原有的内容被截断, 输出: Hello,how  
System.out.println(buf.toString());
```

- length: 当前字符个数
- capacity: 不另外分配内存可以存放的字符个数

练习



StringBuffer提供有一个insert()方法，可以在特定的位置插入特定的数据。

请自己编写代码掌握这一方法的使用，然后，完成以下编程任务：

有一个字符串缓冲区中，已经保存了1~100个整数，但这些整数没有使用分隔符隔开，是像这个样子的——“1234...9899100”，请你写一个程序，将它用“|”分隔为“1|2|3|4|...|100”这种形式。

多知道一点：StringBuilder类



除了StringBuffer类之外，JDK 1.5 新增一个StringBuilder类，它的功能与StringBuffer一样，只不过StringBuffer是线程安全的。



由于不需要加锁，所以StringBuilder性能比StringBuffer要高，但在多线程环境下一定要小心使用。

再学一招: StringTokenizer类

使用StringTokenizer类可以使用指定的定界符将一个字符串分割为多个token。

```
public class UseStringTokenizer {  
    public static void main(String[] args) {  
        System.out.print("请输入一句英文语句: ");  
        var scanner = new Scanner(System.in);  
        var userInput :String = scanner.nextLine();  
  
        StringTokenizer tokens =  
            new StringTokenizer(userInput);  
        StringBuffer output = new StringBuffer();  
        output.append("\nNumber of elements: " +  
            tokens.countTokens() +  
            "\nThe tokens are:\n");  
        while (tokens.hasMoreTokens())  
            output.append(tokens.nextToken() + "\n");  
  
        System.out.println(output.toString());  
    }  
}
```

token是语法分析的基本单位，以“定界符（delimiter）”区分。

请输入一句英文语句: *Hello,how are you?*

Number of elements: 3

The tokens are:

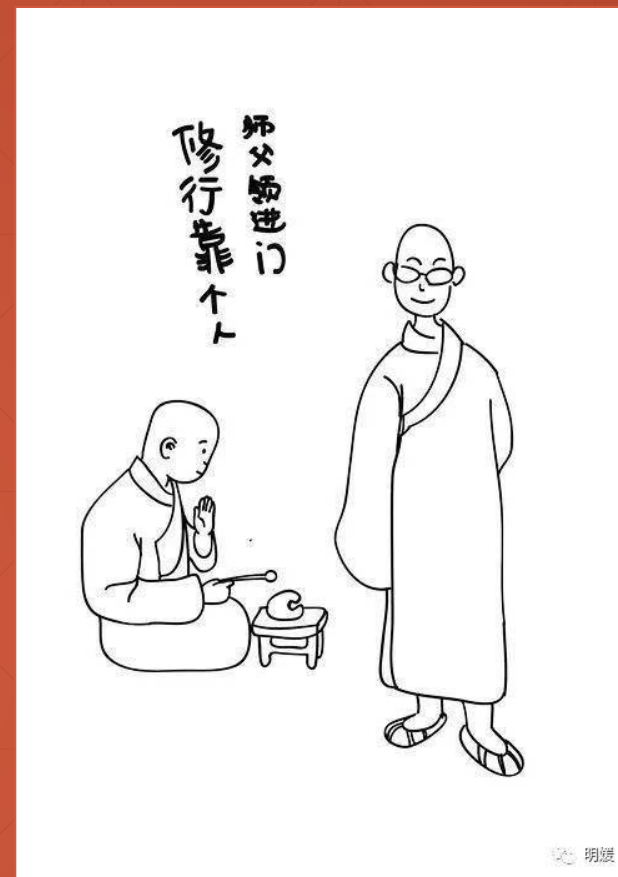
Hello,how

are

you?

使用这个类，可以简化一些有着特定格式的数据文件的处理工作。

课后练习



动手动脑：实现方法的级联调用



与课程中介绍的StringBuffer类似，String类的方法可以也连续调用：

```
String str = "abc";  
String result = str.trim().toUpperCase().concat("defg");
```

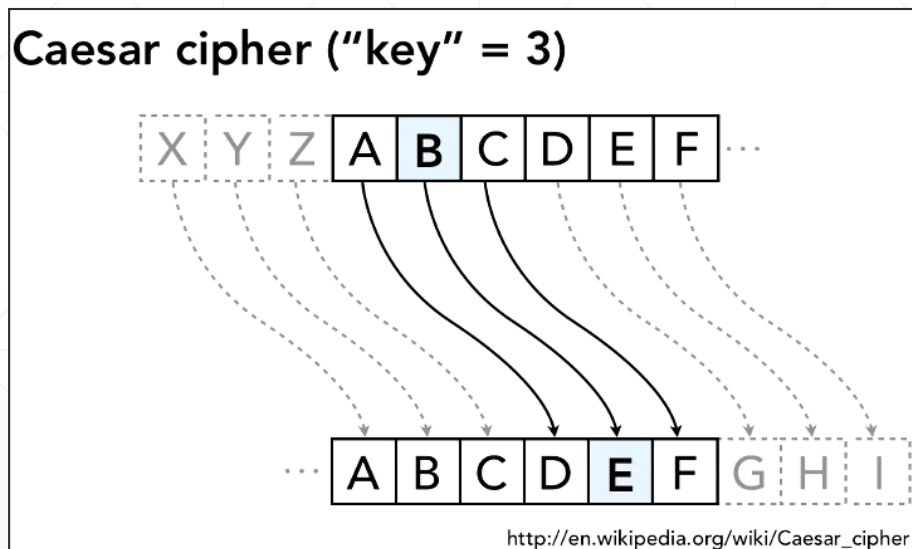
请阅读JDK中String类上述方法的源码，模仿其编程方式，编写一个MyCounter类，它的方法也支持上述的“级联”调用特性，其调用示例为：

```
MyCounter counter1=new MyCounter(1);  
MyCounter counter2=counter1.increase(100).decrease(2).increase(3);  
...
```

动手动脑：字串加密



古罗马皇帝凯撒在打仗时曾经使用过以下方法加密军事情报：



请编写一个程序，使用上述算法加密或解密用户输入的英文字符串。

编程挑战: Email字符串检测



在各种应用程序中，我们经常需要检测用户是否输入了正确的Email地址：

```
JinXuLiang@bit.edu.cn
```

编写一个小工具，它从命令行参数中接收一个Email字符串，判断是否是有效的Email地址，更进一步地，针对不合法的Email字符串，能否给出更具体的信息，指出它哪儿错了？

技术探索

当要处理字符串时，我们既可以用String类，也可以用StringBuilder/StringBuffer类，那么，到底在何种场景下使用String类，何种场景下使用StringBuilder、StringBuffer类？

假设一个开发场景：

你需要在一个比较大的循环（比如循环10000次）中进行字符串的连接、截断等操作，使用上述哪个类合适？性能有差异吗？你能不能写出测试代码证明你的结论？

提示：互联网上可以找到这个问题的答案。



下一讲，我们介绍实际开发中经常要用到的数据结构——数组。